

 Lab Section 1-2

Description Submission view

 Available from: Perşembe, 5 Aralık 2024, 9:10 AM

 Due date: Perşembe, 5 Aralık 2024 10:55 AM

☒ **Due date:** Perşembe, 5 Aralık 2024, 10:55

Requested files: Lab8.java (📄)

Type of work: Individual work
Reduction by automatic evaluation: 5 **Free evaluations:** 5 **Reduction by automatic evaluation:** 5 **Free evaluations:** 5

Java Programming Lab: Matrix Operations and Mathematical Concepts

Lab Purpose

The purpose of this lab is to enhance your understanding of matrix operations, perfect numbers, and Toeplitz matrices using the Java programming language. You will implement specific methods within the `Lab8` class and validate their functionality through provided input-output examples.

Tasks

size **Method**

- Method Name: `size`
- Parameters:
 - `int[][] matrix1` : A two-dimensional integer array representing the first matrix.
 - `int[][] matrix2` : A two-dimensional integer array representing the second matrix.
- Return Type: `int`
- Description: This method calculates and returns the product of the number of rows in `matrix1` and the number of columns in `matrix2`. This value can be used to determine the size of the resulting matrix when performing matrix multiplication.
- Input-Output Examples:
 - Example 1:
 - Input:
 - `matrix1 = [[1, 2], [3, 4], [5, 6]]`
 - `matrix2 = [[7, 8, 9], [10, 11, 12]]`
 - Output: 6
 - Explanation: `matrix1` has 3 rows and `matrix2` has 3 columns. Therefore, $3 * 3 = 9$ should be returned. (Note: *Ensure your implementation correctly reflects the intended calculation.*)
 - Example 2:
 - Input:
 - `matrix1 = [[1]]`
 - `matrix2 = [[2]]`
 - Output: 1
 - Explanation: `matrix1` has 1 row and `matrix2` has 1 column. Therefore, $1 * 1 = 1$ is returned.

— **1998** — **1999** — **2000** — **2001** — **2002** — **2003** — **2004** — **2005** — **2006** — **2007** — **2008** — **2009** — **2010** — **2011** — **2012** — **2013** — **2014** — **2015** — **2016** — **2017** — **2018** — **2019** — **2020** — **2021** — **2022** — **2023** — **2024** — **2025** — **2026** — **2027** — **2028** — **2029** — **2030** — **2031** — **2032** — **2033** — **2034** — **2035** — **2036** — **2037** — **2038** — **2039** — **2040** — **2041** — **2042** — **2043** — **2044** — **2045** — **2046** — **2047** — **2048** — **2049** — **2050** — **2051** — **2052** — **2053** — **2054** — **2055** — **2056** — **2057** — **2058** — **2059** — **2060** — **2061** — **2062** — **2063** — **2064** — **2065** — **2066** — **2067** — **2068** — **2069** — **2070** — **2071** — **2072** — **2073** — **2074** — **2075** — **2076** — **2077** — **2078** — **2079** — **2080** — **2081** — **2082** — **2083** — **2084** — **2085** — **2086** — **2087** — **2088** — **2089** — **2090** — **2091** — **2092** — **2093** — **2094** — **2095** — **2096** — **2097** — **2098** — **2099** — **2100** — **2101** — **2102** — **2103** — **2104** — **2105** — **2106** — **2107** — **2108** — **2109** — **2110** — **2111** — **2112** — **2113** — **2114** — **2115** — **2116** — **2117** — **2118** — **2119** — **2120** — **2121** — **2122** — **2123** — **2124** — **2125** — **2126** — **2127** — **2128** — **2129** — **2130** — **2131** — **2132** — **2133** — **2134** — **2135** — **2136** — **2137** — **2138** — **2139** — **2140** — **2141** — **2142** — **2143** — **2144** — **2145** — **2146** — **2147** — **2148** — **2149** — **2150** — **2151** — **2152** — **2153** — **2154** — **2155** — **2156** — **2157** — **2158** — **2159** — **2160** — **2161** — **2162** — **2163** — **2164** — **2165** — **2166** — **2167** — **2168** — **2169** — **2170** — **2171** — **2172** — **2173** — **2174** — **2175** — **2176** — **2177** — **2178** — **2179** — **2180** — **2181** — **2182** — **2183** — **2184** — **2185** — **2186** — **2187** — **2188** — **2189** — **2190** — **2191** — **2192** — **2193** — **2194** — **2195** — **2196** — **2197** — **2198** — **2199** — **2200** — **2201** — **2202** — **2203** — **2204** — **2205** — **2206** — **2207** — **2208** — **2209** — **2210** — **2211** — **2212** — **2213** — **2214** — **2215** — **2216** — **2217** — **2218** — **2219** — **2220** — **2221** — **2222** — **2223** — **2224** — **2225** — **2226** — **2227** — **2228** — **2229** — **2230** — **2231** — **2232** — **2233** — **2234** — **2235** — **2236** — **2237** — **2238** — **2239** — **2240** — **2241** — **2242** — **2243** — **2244** — **2245** — **2246** — **2247** — **2248** — **2249** — **2250** — **2251** — **2252** — **2253** — **2254** — **2255** — **2256** — **2257** — **2258** — **2259** — **2260** — **2261** — **2262** — **2263** — **2264** — **2265** — **2266** — **2267** — **2268** — **2269** — **2270** — **2271** — **2272** — **2273** — **2274** — **2275** — **2276** — **2277** — **2278** — **2279** — **2280** — **2281** — **2282** — **2283** — **2284** — **2285** — **2286** — **2287** — **2288** — **2289** — **2290** — **2291** — **2292** — **2293** — **2294** — **2295** — **2296** — **2297** — **2298** — **2299** — **2300** — **2301** — **2302** — **2303** — **2304** — **2305** — **2306** — **2307** — **2308** — **2309** — **2310** — **2311** — **2312** — **2313** — **2314** — **2315** — **2316** — **2317** — **2318** — **2319** — **2320** — **2321** — **2322** — **2323** — **2324** — **2325** — **2326** — **2327** — **2328** — **2329** — **2330** — **2331** — **2332** — **2333** — **2334** — **2335** — **2336** — **2337** — **2338** — **2339** — **2340** — **2341** — **2342** — **2343** — **2344** — **2345** — **2346** — **2347** — **2348** — **2349** — **2350** — **2351** — **2352** — **2353** — **2354** — **2355** — **2356** — **2357** — **2358** — **2359** — **2360** — **2361** — **2362** — **2363** — **2364** — **2365** — **2366** — **2367** — **2368** — **2369** — <

- **Method Name:** `multiply`
- **Parameters:**
 - `int[][] matrix1` : A two-dimensional integer array representing the first matrix.
 - `int[][] matrix2` : A two-dimensional integer array representing the second matrix.
- **Return Type:** `int[][]`
- **Description:** This method performs matrix multiplication between `matrix1` and `matrix2` and returns the resulting matrix. Ensure that the number of columns in `matrix1` matches the number of rows in `matrix2` before performing multiplication.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$

$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

- Input-Output Examples:
 - Example 1:
 - Input:
 - $\text{matrix1} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
 - $\text{matrix2} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$
 - Output: $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$
 - Explanation:
 - First row of result: $(1*5 + 2*7) = 19$, $(1*6 + 2*8) = 22$
 - Second row of result: $(3*5 + 4*7) = 43$, $(3*6 + 4*8) = 50$
 - Example 2:
 - Input:
 - $\text{matrix1} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$
 - $\text{matrix2} = \begin{bmatrix} 3 \end{bmatrix}$
 - Output: $\begin{bmatrix} 6 \end{bmatrix}$

isPerfect Method

- **Method Name:** `isPerfect`
- **Parameters:**
 - `int num` : The integer to be checked for being a perfect number.
- **Return Type:** `boolean`
- **Description:** This method determines whether the provided integer `num` is a perfect number. A perfect number is a positive integer that is equal to the sum of its proper positive divisors, excluding itself.
- **Input-Output Examples:**
 - **Example 1:**
 - **Input:** 6
 - **Output:** `true`
 - **Explanation:** The divisors of 6 are 1, 2, and 3. $1 + 2 + 3 = 6$, so 6 is a perfect number.
 - **Example 2:**
 - **Input:** 28
 - **Output:** `true`
 - **Explanation:** The divisors of 28 are 1, 2, 4, 7, and 14. $1 + 2 + 4 + 7 + 14 = 28$, so 28 is a perfect number.
 - **Example 3:**
 - **Input:** 12
 - **Output:** `false`
 - **Explanation:** The divisors of 12 are 1, 2, 3, 4, and 6. $1 + 2 + 3 + 4 + 6 = 16$, which is not equal to 12.
 - **Example 4:**
 - **Input:** 0
 - **Output:** `false`
 - **Explanation:** Zero is not considered a perfect number.
 - **Example 5:**
 - **Input:** -6
 - **Output:** `false`
 - **Explanation:** Negative numbers cannot be perfect numbers.

findPerfectNumbers Method

- **Method Name:** `findPerfectNumbers`
- **Parameters:**
 - `int n` : The number of perfect numbers to find.
- **Return Type:** `String`
- **Description:** This method finds the first `n` perfect numbers and returns them as a space-separated string. If `n` is zero or negative, the method returns an empty string.
- **Input-Output Examples:**
 - **Example 1:**
 - **Input:** `2`
 - **Output:** `6 28`
 - **Explanation:** The first two perfect numbers are 6 and 28.
 - **Example 2:**
 - **Input:** `1`
 - **Output:** `6`
 - **Explanation:** The first perfect number is 6.
 - **Example 3:**
 - **Input:** `0`
 - **Output:** `""`
 - **Explanation:** Requesting zero perfect numbers returns an empty string.
 - **Example 4:**
 - **Input:** `-3`
 - **Output:** `""`
 - **Explanation:** Negative input for `n` results in an empty string.
 - **Example 5:**
 - **Input:** `3`
 - **Output:** `6 28 496`

- **Method Name:** `isToeplitzMatrix`
- **Parameters:**
 - `int[][] matrix` : A two-dimensional integer array representing the matrix to be checked.
- **Return Type:** `boolean`
- **Description:** This method checks whether the provided matrix is a Toeplitz matrix. A Toeplitz matrix is one where each descending diagonal from left to right is constant. An empty matrix or a matrix with a single element is automatically considered Toeplitz.

$$\bullet \quad A = \begin{bmatrix} a_0 & a_{-1} & a_{-2} & \cdots & \cdots & a_{-(n-1)} \\ a_1 & a_0 & a_{-1} & \ddots & & \vdots \\ a_2 & a_1 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & a_{-1} & a_{-2} \\ \vdots & & \ddots & a_1 & a_0 & a_{-1} \end{bmatrix}.$$

- **Input-Output Examples:**
 - **Example 1:**
 - **Input:** `[[1, 2, 3], [4, 1, 2], [5, 4, 1]]`
 - **Output:** `true`
 - **Explanation:** Each descending diagonal has identical elements.
 - **Example 2:**
 - **Input:** `[[1, 2], [2, 3]]`
 - **Output:** `false`
 - **Explanation:** The diagonal starting at (1,1) has different elements (2 vs. 3).
 - **Example 3:**
 - **Input:** `[[5, 4, 3, 4], [2, 5, 4, 3], [1, 2, 5, 4]]`
 - **Output:** `true`
 - **Explanation:** Each descending diagonal has identical elements.
 - **Example 4:**
 - **Input:** `[[42]]`
 - **Output:** `true`
 - **Explanation:** A single-element matrix is Toeplitz by definition.
 - **Example 5:**
 - **Input:** `[]`
 - **Output:** `true`
 - **Explanation:** An empty matrix is considered Toeplitz.
 - **Example 6:**
 - **Input:** `null`
 - **Output:** `true`
 - **Explanation:** A null matrix is treated as an empty matrix and thus Toeplitz.